HPPS 2024
Assignment four
Locality Optimizations

Marcus Frehr Friis-Hansen lns611
Noah Wenneberg Junge qxk266
Group 7

December 19, 2024

Introduction

Disclaimer: These programs were developed with chatgbt, primarily to verify syntax and debug the code. To run the code first type `Make all` in the terminal while being in the correct register. Then download the dataset using `make planet-latest-geonames.tsv`. Then to test on smaller subsets of the data extract the first eg. 20000 samples like so: `head -n 20000 planet-latest_geonames.tsv > 20000records.tsv`. Then run each of the programs with the commands shown below.

```
////to run the first program on 5000 samples
    ./id_query_naive 5000records.tsv

/// example
65606
65606: London -0.144055 51.489333
Query time: 2us


////to run the second program on 50 000 samples
    ./id_query_indexed 50000records.tsv


////to run the third program on 500 000 samples
    ./id_query_binsort 500000records.tsv


////to run the fourth program on 500 000
./coord_query_naive 500000records.tsv

-0.1440551      51.489333
(-0.144055,51.489333): London (-0.144055,51.489333)
Query time: 13024us
```

3.1 id_query_naive.c

The program loads the dataset into memory, and if instructed to using a query, it performs linear search to locate a specific record based on `osm\_id`.

`mk_naive` initializes and allocates memory for the naive_data structure, it allocates memory for the array of records, and then copies these records into the allocated memory. `free_naive` frees the memory for the allocated records, and then frees up the `naive_data` structure itself. `lookup_naive` performs a linear search using a loop to iterate through the array of records. If found, it returns a pointer to the record, using the o`osm_id` as reference, if not it returns `NULL`

The code is functionally correct, however it is a bit slow since it uses linear search. using a different search algorithm like binary search would be faster, but that would also require the data to be sorted.

3.2 id_query_indexed.c

The idea behind this program is that instead of searching through the actual data structure, we search through a lightweight version of it, retaining only the `osm_id` and a pointer to said entry in the original dataset. This should give us faster querying

`mk_indexed` works by allocating memory for the `indexed_data` structure, then `indexed_record` which are the data entries. If all goes well it loads the data entries and returns a pointer to the data structure. `free_indexed` frees the `indexed_record`s, followed by freeing the data records themselves.`lookup_indexed` simply performs a linear search on the `indexed_data` structure using a for-loop, return a pointer to the match if successful.

The code is functionally correct and should be faster than `id_query_naive`, even though they both use linear search, as it searches a more more sparse data structure.

3.3 id_query_binsort.c

This program builds upon the same idea of using a lightweight data structure for faster querying, but adds the benefit of a sorted array and binary search for added efficiency. This results in a runtime of $O(\log(n))$, which is faster when compared to the runtime of the earlier implementations which were $O(n)$. The overall structure of the implementation is quite similar to the other programs, so we wont expand on it any further other than stating it is functionally correct and and avoids data leakage using if statements, and freeing up data in case `malloc` fails to allocate memory.

4. coord_query_naive.c

This program finds the closest record in a dataset to a given pair or coordinates. much like the first program it performs a linear search to go through all records, but does so by keeping track of the minimum distance and updates the closest record during the search. Much like the other programs it is functionally correct, as it computes the euclidean distance correctly for all records and updates properly during the search. The program does not corrupt memory. Yes, it searches through the actual data, but only allocates memory for the `naive_data` structure, which is freed using `free_naive` afterwards. it also doesn't leak any memory for the same reason.

Questions

please note that we use numbered list to refer to each of the programs in order.

Evaluate the temporal and spatial locality of the four programs you have implemented.

1. `id_query_naive`: The temporal locality of the program is poor, as it searches through the entire array for each query, and they are not used again in any subsequent query. The

	Dataset size	Query time
id_query_naive.c	5000	2us
id_query_naive.c	50000	4us
id_query_naive.c	500000	62us
id_query_indexed.c	5000	2us
id_query_indexed.c	50000	2us
id_query_indexed.c	500000	1us
id_query_binsort.c	5000	5us
id_query_binsort.c	50000	3us
id_query_binsort.c	500000	3us
coord_query_naive.c	5000	16767us
coord_query_naive.c	50000	1544us
coord_query_naive.c	500000	13421us

Table 1: Program benchmarks:
London used for each query: osm_id:65606, lon:-0.1440551, lat: 51.4893334

spatial locality is alright since the elements in the array are accessed sequentially.

2. `id_query_indexed`: The temporal locality is better than the earlier implementation. we still go through the data for each query, but the data structure itself is now more lightweight. The spatial locality is again quite alright as the indexes are successive in memory and we access them sequentially.

3. `id_query_binsort`: The temporal locality is good, as binary search reuses parts of the data repeatedly cutting it in half while searching. The spatial locality is also good, as it uses a sorted, indexed array that is contiguous in memory.

4. `coord_query_naive`: The temporal locality is bad, as it goes through the entire dataset for each query. This could be improved by first sorting the array based on longitude and latitude. The spatial locality is good, as the data is stored as a contiguous array, which the program accesses sequentially.

Do your programs corrupt memory? Do they leak memory, and how do you know?

1. `id_query_naive`: There shouldn't be any memory leaks or corruption, as we use `malloc` and `free` to safely allocate and free up memory in case something unexpected happens.`free_naive` frees up the records and the data structure itself after use.

2. `id_query_indexed`: As the program uses and index and pointers to map to the original data, never accessing it directly, data corruption is very unlikely. Data leaks is also unlikely as it uses `free_indexed` to free up the memory used by the indexed array and its entries.

3. `id_query_binsort`: Data corruption or leakage is unlikely, as the actual data isn't manipulated, but only accessed through the pointers in the index array, and that `free_binsort` frees up the sorted index and the memory structure.

4. `coord_query_naive`: The program does not leak or corrupt any data as it allocates it in `mk_naive` using `malloc`, and frees it in `free_naive` using `free`

How confident are you that your programs are correct?

We are confident that the programs are functionally correct. We have gone over the programs multiple times. The fact that the programs compile and execute without error should cement this.

Benchmark your programs on various workloads and explain the differences. How did you pick the workloads?

To benchmark the four programs we selected 3 different dataset sizes, each being ten times larger than the former. The sizes being 5000 records, 50000 records and 500000 records, to best capture the performance of the programs on different size datasets.

1. `id_query_naive` uses linear search, so we expected the query time to increase linearly ($O(n)$) as the dataset grew in size. This is visible in the table above as the query time grows from $2us$ for 5000 words, to $4us$ and $62us$ for 50000 and 500000 records respectively.

2. `id_query_indexed` also uses linear search. However we still expect it to be faster than `id_query_naive` as the number of records increases. This can be seen in the table where the runtime is $5us$, $3us$ and $3us$ for 5000, 50000 and 500000 records respectively. This is caused by the fact that reduced size of the indexed array.

3. `id_query_binsort` uses binary search, so we expected $O(\log n)$ runtime, translating to the runtime increasing minimally as the number of records grow from 5000 all the way up to 500000. This somewhat fits with the observed query times of $2us$, $2us$ and $1us$ for 5000, 50000 and 500000 records respectively.

4. `coord_query_naive` uses linear search to go through the entire dataset for each query. We expect the runtime to increase linearly as the number of records increase, meaning $O(n)$. While the run times are diffidently much longer for this program, when compared to the three others, it is difficult to conclude if the runtime actually scales linearly to the data size. This might be due to how my computer loads the data into cache. The runtime in relation to the data-size can be seen in the table above.